

上海大学 计算机工程与科学学院

《数据库原理》(1) 实验报告

姓名 赵文宇 学号 23121769

时间 一 3-4 地点 计 706 指导教师 李晓强

实验 4: SQL 命令-聚合函数、存在量词和数据更新

一、实验目的

- 1、了解和掌握 SQL 命令中聚合函数的使用
- 2、了解和掌握 SQL 命令中存在量词的使用
- 3、了解和掌握 SQL 命令中数据更新命令的使用

二、实验内容

- 1、lect04 讲稿中的例子练习
- 2、对 school 数据库做如下查询：
 - 1) 建立计算机学院总评不及格成绩学生的视图，包括学生学号、姓名、性别、手机、所选课程和成绩。
 - 2) 在 E 表中插入记录，把每个学生没学过的课程都插入到 E 表中，使得每个学生都选修每门课。
 - 3) 求年龄大于所有女同学年龄的男学生姓名和年龄。
 - 4) 在 E 表中修改 08305001 课程的平时成绩，若成绩小于等于 75 分时提高 5%，若成绩大于 75 分时提高 4%。
 - 5) 删除没有开课的学院。
 - 6) 查询优、良、中、及格、不及格学生人数

三、实验过程

(描述实践的过程和结果、遇到的问题 and 解决方案)

1: 对 lect04 讲稿中例子进行的练习:

(1) 查询使用了全部零件的工程名称

这个例子，课件中给出的 sql 语句是基于是用两层 not exists 的“双重否定表肯定”来解决的，“查询使用了全部零件的工程名称”等价于“查询没有一个零件是没有使用的工程名称”。我先对这个逻辑对应的 sql 语句编写进行实验。语句如下所示:

```

use gyqk
select jno,jname from J
where not exists(
    select * from P
    where not exists(
        select * from SPJ
        where SPJ.jno = J.jno AND
        SPJ.pno = P.pno
    )
)

```

这个逻辑没有太多需要解释的地方，就是因为 sql 中不存在全称量词，只有存在量词，这两者是可以进行相互转换的，可以用双重否定表肯定。但是我在进行实验的时候，明显感觉这样“反过来”思考的过程十分“反直觉”，因此，一开始我的写法并不是这样的，我是将 count() 这个聚合函数放在 having 中进行判断的，具体的 sql 语句如下：

```

use gyqk
select J.jno,J.jname
from J
JOIN SPJ ON J.jno = SPJ.jno
JOIN P ON P.pno = SPJ.pno
GROUP BY J.jno,J.jname
Having count(distinct P.pno) = (select distinct count(*) from P)

```

这里的基础逻辑就是先将 P、J、SPJ 表显示链接到一块，然后再用 group by 将这张链接后的“大表”根据 jno 划分到不同的“抽屉”里（我是这样理解的，感觉比较形象一些），然后左侧的 count(distinct P.pno) 就是代表每一个工程 J 下用了多少种不同的零件，而 (select distinct count(*) from P) 则是 P 表中不同零件的个数，只要两者相等，就代表是用了所有零件的工程。

查询结果如下图所示：

结果		消息
	jno	jname
1	J4	炼钢工厂

(2) 查询至少用了供应商 S1 所供应的全部零件的工程号

这个题目也是一样，将“至少”转化为“不存在……不……”，即“查询不存在一个供应商 S1 提供的零件都没有被使用的工程号”。同样是用一个两层 not exists 进行查询，不过我这边写的语句与课件中给出的查三次 SPJ 表不同，我觉得最外层的 SPJ 表用 J 表替换也是完全可行的，具体的 sql 语句如下所示：

```
use gyqk
select jno from J
where not exists(
    select * from SPJ as a
    where a.sno = 's1' AND not exists(
        select * from SPJ as b
        where b.jno = J.jno
        AND b.pno = a.pno
    )
)
```

这边一开始中间层我是用 select * from P 来写的，但是一查询，发现查询结果为空，经过 gpt 帮助后，我才意识到如果是 from P 的话，逻辑是检测所有 P 表中的零件，与这边只想检测“S1”供应的零件在逻辑上是不一样的，应该改为存放 sno 记录的 SPJ 表才可以。

查询结果如下图所示：

结果		消息
		jno
1	J4	

(3) 查询价格在 0-30, 31-60, 61-80, 81-100 间的平均单价

这个例子重点考察的是对于 union 这个集合并操作的使用，我们可以分别查询价格在 0-30、31-60、61-80、81-100 区间的平均单价，再用 union 将这几个 select 语句连在一起即可。具体的 sql 语句如下：

```
USE gyqk
select '0-30' as qujian,AVG(price) as avg_price from SPJ where price between
0 and 30
union
```

```

select '31-60' as qujian,AVG(price) as avg_price from SPJ where price between
31 and 60

union

select '61-80' as qujian,AVG(price) as avg_price from SPJ where price between
61 and 80

union

select '81-100' as qujian,AVG(price) as avg_price from SPJ where price
between 81 and 100

```

运行结果如下：

结果	消息	
	qujian	avg_price
1	0-30	19.446153
2	31-60	33.800000
3	61-80	NULL
4	81-100	NULL

(4) 查询使用了红色和蓝色零件的工程号

这个例子重点考察的是对于 intersect 这个集合交操作的使用，我们可以分别查询使用了红色零件/蓝色零件的工程号，再用 intersect 将这两个 select 语句连在一起即可表示同时使用了红色零件和蓝色零件。具体的 sql 语句如下：

```

USE gyqk

select distinct jno from SPJ,P where SPJ.pno=P.pno and P.color='红'

intersect

select distinct jno from SPJ,P where SPJ.pno=P.pno and P.color='蓝'

```

但是可能因为记录中不存在这样的工程，运行结果为空。

(5) 在 J 表中插入一个元组('J8' , '地铁两号线' , '上海')

就是一个比较简单的记录插入语句，直接编写即可， sql 语句如下：

```

USE gyqk

insert into j(jno,jname,jcity) values('J8','地铁两号线','上海')

```

运行如果如下：

(1 行受影响)

完成时间： 2026-04-08T11:28:14.9254019+08:00

(6) 插入工程号为 J7 的供应零件记录，供应给 J7 的零件和供应给 J1 的

零件号及供应商相同，但每种零件的供应量改为 60

把 values 用 select 语句替换即可，select 语句就是对于条件的查询（把上面的陈述句看成是对于满足这些条件的记录的查询即可）。具体 sql 语句如下：

```
USE gyqk
insert into SPJ(sno,pno,jno,qty)
select sno,pno,'J7',60 from SPJ where jno = 'J1'
```

运行结果如下图所示：

(5 行受影响)

完成时间: 2026-04-08T11:32:16.7557465+08:00

(7) 当 S3 提供的 P5 零件的单价低于该零件的平均单价时，将其提高 8%

更新数据，用 update 即可。这边还有更新条件，所以在 update 语句中需要增加 where 条件判断，这边的逻辑和 select 中的 where 是相同的。

具体的 sql 语句如下：

```
USE gyqk
update spj set price=price*1.08
where sno='S3' and pno='P5'
and price<(select AVG(price) from spj where pno='P5')
```

2(1) 建立计算机学院总评不及格成绩学生的视图，包括学生学号、姓名、性别、手机、所选课程和成绩

这个例子需要用到 S、D、E、C 四张表，筛选条件为学院名称为“计算机学院”且总评成绩不及格。

代码如下：

```
--USE school;
CREATE VIEW Fail_Stu as
SELECT S.xh as 学号, S.xm as 姓名, S.xb as 性别, S.sjhm as 手机, C.km as 课程,
E.zpcj as 成绩
FROM S
JOIN D ON S.yxh = D.yxh
JOIN E ON S.xh = E.xh
JOIN C ON E.kh = C.kh
WHERE D.mc = '计算机学院' AND E.zpcj < 60;
```

2) 在 E 表中插入记录, 把每个学生没学过的课程都插入到 E 表中, 使得每个学生都选修每门课。

只需要在所有学生与所有课程的总笛卡尔积中删掉那些那些已经存在于选课表中的组合, 即可查到对应的记录。

代码如下:

```
USE school

INSERT INTO E (xh, xq, kh, gh)

SELECT S.xh, O.xq, C.kh, O.gh FROM S

CROSS JOIN C

JOIN O ON C.kh = O.kh

WHERE NOT EXISTS (

    SELECT * FROM E WHERE E.xh = S.xh AND E.kh = C.kh

)
```

运行结果如下:

(29 行受影响)

完成时间: 2026-04-08T13:43:42.0775372+08:00

3) 求年龄大于所有女同学年龄的男学生姓名和年龄。

找出来所有女同学的最大年龄, 然后找出年龄大于此值的男同学, 即可查找到对应的记录。

代码如下:

```
USE school

SELECT S.xm as 姓名,

    DATEDIFF(YEAR, S.csrq, GETDATE()) as 年龄

FROM S

WHERE S.xb = '男' AND DATEDIFF(YEAR, S.csrq, GETDATE()) > ( SELECT

    MAX(DATEDIFF(YEAR, csrq, GETDATE()))

    FROM S WHERE xb = '女'

);
```

运行结果如下:

结果		消息
	姓名	年龄
1	刘晓明	34
2	刘成刚	35
3	张晓峰	34

4) 在 E 表中修改 08305001 课程的平时成绩，若成绩小于等于 75 分时提高 5%，若成绩大于 75 分时提高 4%。

使用 CASE 表达式进行条件判断，根据不同的成绩区间，对平时成绩进行不同幅度的提高。

代码如下：

```
USE school
UPDATE E
SET pscj = ROUND(
    CASE
        WHEN pscj <= 75 THEN pscj * 1.05
        ELSE pscj * 1.04
    END, 0)
WHERE kh = '08305001' AND pscj IS NOT NULL
```

运行结果如下：

(6 行受影响)

完成时间：2026-04-08T13:55:07.0635158+08:00

5) 删除没有开课的学院。

最内层子查询通过 EXISTS 判断课程是否在开课表中存在。中间子查询找出所有有开课记录的课程的所属院系编号。最后，删除不在这个院系编号列表中的学院记录。

代码如下：

```
DELETE FROM D
WHERE yxh NOT IN (
    SELECT DISTINCT C.yxh FROM C
    WHERE EXISTS (
        SELECT * FROM O
```

```
WHERE O.kh = C.kh

)

);
```

6) 查询优、良、中、及格、不及格学生人数

根据总评成绩进行等级划分，然后统计每个等级的学生人数，最后即可查到对应的记录。

代码如下：

```
USE school

SELECT
    成绩等级, COUNT(*) AS 学生人数
FROM (
    SELECT CASE
        WHEN E.zpcj >= 90 THEN '优'
        WHEN E.zpcj >= 80 THEN '良'
        WHEN E.zpcj >= 70 THEN '中'
        WHEN E.zpcj >= 60 THEN '及格'
        ELSE '不及格'
    END AS 成绩等级
    FROM E
    WHERE E.zpcj IS NOT NULL
) AS GradeTable

GROUP BY 成绩等级

ORDER BY CASE 成绩等级
    WHEN '优' THEN 1
    WHEN '良' THEN 2
    WHEN '中' THEN 3
    WHEN '及格' THEN 4
    ELSE 5

END
```

运行结果如下：

结果 消息		
	成绩等级	学生人数
1	优	4
2	良	4
3	中	4
4	及格	2
5	不及格	1

四、收获与感想

这次实验四的内容比之前丰富不少，搞定了聚合函数、存在量词和一堆数据更新操作，感觉对 SQL 的操控力强了很多。最大的收获是搞明白了怎么用 GROUP BY 和 HAVING 来分组统计，更关键的是，终于把那个很绕的 EXISTS/NOT EXISTS 子查询给整明白了——原来“查询使用了全部零件”这种问题，既可以像课件里那样用两层 NOT EXISTS（双重否定表肯定）来“绕”出来，也可以用 COUNT() 在 HAVING 里判断相等来解决。虽然前者有点“反直觉”，但两种方法都试过后，对全称量词怎么转换成 SQL 逻辑清晰多了。

另外，这次实战了各种数据操作，比如用 CASE WHEN 根据不同条件更新成绩、用 INSERT ... SELECT 和 CROSS JOIN 给所有学生批量插入没学过的课，还有删除没开课的学院。特别是最后那个“让每个学生都选上每门课”的插入任务，从想清楚逻辑（笛卡尔积去掉已选的）到写出代码，中间还调试了一下，完整走通一遍后，对多表关联和子查询配合使用的感觉实在多了。

总的来说，实验四让我对 SQL 的认知从“会写简单查询”往前跨了一步，开始能用它来处理更实际的业务逻辑了，算是朝着“更熟练地折腾数据库”又迈进了一小步吧。==